

FleetLink frontend API reference

****Source and scope.**** The exported `FleetLink API` Postman collection is now the source of truth for the ****28 configured requests**** in this document. Bodies, query parameters, local base URL, and collection variables below were reconciled with the current FleetLink routes/controllers. Two empty Postman placeholders (`Fleet Management/Vehicle Maintenance / New Request` and `Tenant Package & Entitlement Management / New Request`) are not endpoints and are excluded.

Postman collection environment

The collection has no reusable baseUrl variable; every configured request targets http://localhost:3000. It declares the following ****secret**** variables, but the export deliberately contains no values for them:

Postman variable • Used by configured requests • Frontend meaning
bearer_token_0esg • Fleet dashboard • JWT for a Manager/Admin user.
supabase_service_role_api_key_0u0w • Register vehicle, update odometer, log service record • JWT for a Manager/Admin user. The name is misleading: FleetLink verifies a JWT signed with JWT_SECRET, not a Supabase service key by itself.
json_web_token_0u0w • Service history, approaching maintenance • JWT for a Manager/Admin user.
bearer_token_0aqh • Availability and booking flow • JWT for a Staff/Manager/Admin user; approval/rejection requires Department Head/Manager/Admin.
json_web_token_0da4 • Customer and rental flow • Any valid FleetLink JWT.
supabase_service_role_api_key_151n • Analytics, reports, audit logs • JWT for a Manager/Admin user.
json_web_token_07d8, bearer_token_0hp0, json_web_token_15ut, bearer_token_19ec, bearer_token_02is • Not referenced by a saved request • Legacy/unused collection secrets; remove or document their intended use.

Set every protected request to Authorization: Bearer {{your_token_variable}}. The exported dashboard, availability, and booking requests use a bare variable in their saved header; it must resolve to the complete Bearer <JWT> value or be changed to the standard header format above.

Tested-request inventory

Collection folder • Requests configured
Tenants • Create Tenant
Departments • Create Department
Auth • Sign up; Login
Fleet Management / Vehicle Maintenance • Get Fleet Dashboard; Register new Vehicle; Update Odometer Reading; Log Vehicle Service Record; List/Get Service History; Get Vehicle Approaching Maintenance; Find Available Vehicles; Submit Booking Request; Approve Corporate Booking; Reject Corporate Booking
Customer Management (Rentals) • Create Rental Customer
Rental Reservations & Inspection Flow • Create Rental Reservation; Perform Checkout Inspection; Extend Rental Duration; Perform Checkin Inspection
Analytics & Efficiency Metrics • Vehicle Utilization; Department ROI; Top Requesters; Fuel & Energy Consumption; Maintenance Compliance
Reports & Scheduled Exports • View Rental Performance; Export Rental Performance File; Schedule Emailed Rental Performance Report
Security & Audit Logging • Query Audit Logs

Run and configure

Start the FleetLink API with npm start. It listens on http://localhost:3000 unless PORT is set. Do ****not**** use npm run dev for this API at present: it starts the separate TypeScript project demo on port 4000.

Use these frontend environment values:

```
VITE_FLEETLINK_API_URL=http://localhost:3000
# or for Next.js:
NEXT_PUBLIC_FLEETLINK_API_URL=http://localhost:3000
```

Backend runtime variables:

Variable • Required • Purpose
DATABASE_URL • Yes • PostgreSQL connection used by Prisma and background jobs.
PORT • No • API port; defaults to 3000.
JWT_SECRET • Yes in deployed environments • Token signing/verification secret. A development fallback exists and must not be used in production.
JWT_EXPIRES_IN • No • JWT lifetime; defaults to 7d.
CLOUDINARY_CLOUD_NAME, CLOUDINARY_API_KEY, CLOUDINARY_API_SECRET • Required only for vehicle image upload • Cloudinary upload configuration.
SMTP_HOST, SMTP_PORT, SMTP_SECURE, SMTP_USER, SMTP_PASS, SMTP_FROM • Required for notification/report email delivery • SMTP configuration.

All JSON requests use Content-Type: application/json. Protected routes also require:

Authorization: Bearer <token returned by POST /auth/login>

Dates must be ISO-8601 values with a timezone, for example 2026-08-25T09:00:00.000Z. IDs below are UUIDs. Monetary and decimal values should be sent as strings (for example, "125.00") to avoid client-side precision surprises.

Authentication and setup

Method • Endpoint • Auth / role • Request body • Success

POST • /tenants • None • { "name", "sector", "billingStatus", "package"?, "featureOverrides"? } • 201 { message, tenant }
POST • /departments • None • { "tenantId", "name", "costCentreCode"?, "budgetCode"? } • 201 { message, department }
POST • /auth/signup • None • { "name", "email", "password", "role", "tenantId", "departmentId"?, "contact"?, "notificationPreferences"? } • 201 { message, user }
POST • /auth/login • None • { "email", "password" } • 200 { token, user: { id, name, role, tenantId } }
GET • /fleet/dashboard • FLEET_MANAGER, SUPER_ADMIN • — • 200 { message, tenantId }

role is stored as a string. Roles used by guards are STAFF, DEPARTMENT_HEAD, FLEET_MANAGER, and SUPER_ADMIN.

****Security note:**** tenant and department creation plus signup are currently public. The frontend must not expose those setup routes as ordinary user actions until backend access control is added.

Vehicles and maintenance

Method • Endpoint • Role • Params/query • Request body • Success
POST • /fleet/vehicles • Manager/Admin • — • JSON: { "registration", "make"?, "model"?, "departmentId"?, "odometerCurrent"?, "imageUrl"? }; or multipart/form-data with the same fields plus image file field image • 201 { vehicle }
PATCH • /fleet/vehicles/:vehicleId/image • Manager/Admin • Path: vehicleId • { "imageUrl": "https://..." } • 200 { vehicle }
PATCH • /fleet/vehicles/:vehicleId/odometer • Manager/Admin • Path: vehicleId • { "odometerCurrent": 12500 } • 200 { vehicle, maintenance }
POST • /fleet/vehicles/:vehicleId/service-records • Manager/Admin • Path: vehicleId • { "serviceDate", "odometerMileage", "cost", "description"?, "nextDueDate"?, "nextDueMileage"? } • 201 { serviceRecord, warnings, maintenance }
GET • /fleet/vehicles/:vehicleId/service-records • Manager/Admin • Path: vehicleId • — • 200 { serviceRecords }
GET • /fleet/maintenance/approaching • Manager/Admin • Query: days? (default 30), mileage? (default 1000) • — • 200 { vehicles }
GET • /api/vehicles/available • Staff/Manager/Admin • Query: start, end (both required ISO timestamps) • — • 200 { window: { start, end }, vehicles, excludedVehicleCount }

Vehicle rules: registration is required and unique per tenant; odometerCurrent is a non-negative integer and cannot be decreased. Image uploads accept image MIME types up to 5 MB; accepted Cloudinary formats are JPG, JPEG, PNG, and WebP. imageUrl must be an HTTP(S) URL.

The Postman collection tests every row in this section except ****Update vehicle image****; that endpoint is source-documented for completeness.

Corporate bookings

Method • Endpoint • Role • Params • Request body • Success
POST • /api/bookings • Staff/Manager/Admin • — • { "vehicleId", "start", "end", "justification", "passengerCount", "kind"? } • 201 { booking }
POST • /api/bookings/:bookingId/approval • Department Head/Manager/Admin • Path: bookingId • No body • 200 { booking }
POST • /api/bookings/:bookingId/rejection • Department Head/Manager/Admin • Path: bookingId • { "comment" } • 200 { booking }

kind is CORPORATE (default) or RENTAL; passengerCount is a positive whole number. start must precede end. A request starts as PENDING; only its assigned approver, a Fleet Manager, or a Super Admin can action it. A vehicle cannot be approved for overlapping time slots. Booking results use startAt and endAt in UTC ISO format.

Rental flow

Recommended frontend sequence: create customer !' create reservation !' checkout !' optional extension !' checkin.

Method • Endpoint • Role • Params • Request body • Success
POST • /api/customers • Any authenticated user • — • { "name", "email", "driverLicense", "contact"? } • 201 { customer }
POST • /api/rental-reservations • Any authenticated user • — • { "vehicleId", "customerId", "start", "end", "agreedRate", "depositAmount", "depositPaid"?, "rentalAgreement"? } • 201 { reservation }
POST • /api/rental-reservations/:reservationId/extend • Any authenticated user • Path: reservationId • { "newEnd": "<ISO timestamp>" } (end is also accepted) • 200 { reservation }
POST • /api/rental-reservations/:reservationId/inspections/checkout • Any authenticated user • Path: reservationId • [inspection payload](#inspection-payload) • 201 { reservation }
POST • /api/rental-reservations/:reservationId/inspections/checkin • Any authenticated user • Path: reservationId • [inspection payload](#inspection-payload) • 201 { reservation }

Inspection payload

```
{
  "odometerReading": "12500.00",
  "fuelLevel": "85.00",
  "chargeLevel": "60.00",
  "conditionNotes": "No new damage",
  "conditionPhotos": ["https://cdn.example.com/inspection-front.jpg"]
}
```

odometerReading is required and non-negative. fuelLevel and chargeLevel are optional non-negative decimals.

conditionPhotos must be an array of URLs/strings.

- Checkout is allowed only from `RESERVED` and requires at least one photo; it moves the reservation to `ACTIVE`.
- Checkin is allowed from `ACTIVE` or `OVERDUE`, requires an odometer no lower than checkout, and moves it to `COMPLETED`. Photos are required unless notes explicitly confirm no new damage (for example, `"No new damage")`).
- A reservation cannot overlap another `RESERVED`, `ACTIVE`, or `OVERDUE` rental for the same vehicle. A customer with an overdue rental cannot make another reservation. Extensions must move the deadline later and cannot conflict.

Reservation status values are RESERVED, ACTIVE, COMPLETED, and OVERDUE.

Analytics, reports, and audit log

All of these routes require a Manager or Super Admin token. Date filters are optional ISO timestamps.

Method • Endpoint • Extra access • Query/body • Success

GET • /api/analytics/vehicle-utilization • advancedAnalytics entitlement • Query: start?, end? • 200 { metrics }

GET • /api/analytics/departments-roi • advancedAnalytics entitlement • Query: start?, end? • 200 { metrics }

GET • /api/analytics/top-requesters • advancedAnalytics entitlement • Query: start?, end?, limit? (default 10) • 200 { metrics }

GET • /api/analytics/fuel-efficiency • — • Query: start?, end? • 200 { report }

GET • /api/analytics/maintenance-compliance • — • Query: start?, end? • 200 { report }

GET • /api/reports/performance • — • Query: start?, end?, currency? (default USD), locale? (default en-US) • 200 { report }

GET • /api/reports/performance/export • — • Same filters plus format=pdf (default) or format=xlsx • File stream with download headers

POST • /api/reports/performance/schedule • — • { "recipient"?, "format"?, "currency"?, "locale"?, "start"?, "end"?, "scheduleAt"? } • 200 { scheduled: true, jobId }

GET • /api/audit-logs • — • Query: actorId?, category? (AUDIT/SECURITY), from?, to?, limit? (1–500; default 100) • 200 { auditLogs }

Scheduled reports require background jobs and a recipient; if omitted, the system selects the tenant's active Fleet Manager.

Report scheduling and email delivery therefore also require the background-job database setup and SMTP configuration.

Entitlements: implemented but currently unreachable

The routes below exist in src/routes/entitlement.routes.js but are **not mounted** in src/app.js, so they currently return 404 and should not be integrated yet.

Method • Intended endpoint • Body • Intended access

GET • /api/entitlements/:tenantId? • — • Super Admin

PATCH • /api/entitlements/:tenantId/package • { "package", "featureOverrides"? } • Super Admin

Error handling and frontend guidance

- Authentication failures return `401 { "message": "Unauthorized" }` or `401 { "message": "Invalid token" }`; role failures return `403 { "message": "Forbidden" }`.

- Expected validation errors generally return `400 { "message": "..." }`; missing bookings/vehicles use `404`; an approved-booking collision uses `409`.

- `src/app.js` imports but does not register its custom error middleware. Until `app.use(errorHandler)` is added after routes, unhandled errors may use Express's default response rather than the documented JSON error shape. Frontend code should handle non-JSON errors defensively.

- Tenant identity is always taken from the JWT on protected routes. Do not send a `tenantId` in normal protected request bodies.

Frontend request helper

```
const apiUrl = import.meta.env.VITE_FLEETLINK_API_URL;

export async function fleetFetch(path: string, token: string, init: RequestInit = {}) {
  const response = await fetch(`${apiUrl}${path}`, {
    ...init,
    headers: {
      Authorization: `Bearer ${token}`,
      ...(init.body instanceof FormData ? {} : { "Content-Type": "application/json" }),
      ...init.headers,
    },
  });

  if (!response.ok) {
    const error = await response.json().catch(() => ({ message: response.statusText }));
    throw new Error(error.message || "FleetLink request failed");
  }

  return response;
}
```